

Guia de Contribuição

Web Beauty Parlor Management — Fluxo de trabalho com Git e GitHub

Cada integrante é responsável por um commit específico, mapeado no Plano de Ação. Este guia descreve o fluxo completo: desde preparar o ambiente local até ter o código aceito no repositório oficial.

Branches Principais

Branch	Propósito
master	Código estável e aprovado. Nunca recebe commits diretos.
develop	Branch de integração. Todos os PRs devem ser abertos para cá.

Fluxo: branch pessoal → PR para **develop** → revisão e testes → merge em develop → (quando fase estiver estável) → PR de develop para **master**

Para todos os contribuidores

Passo 0 — Preparação Inicial (apenas na primeira vez)

Antes de escrever qualquer código, traga o projeto para sua máquina e configure o ambiente.

1. Fazer o Fork no GitHub

Acesse <https://github.com/perinotti/web-beautyparlor-management> e clique em Fork no canto superior direito.

2. Clonar o seu Fork

No terminal, substitua SEU_USUARIO pelo seu usuário do GitHub:

3. Entrar na pasta e abrir a IDE

Navegue até a pasta criada e abra seu editor:

4. Criar e ativar o ambiente virtual

Crie o ambiente isolado do Python:

5. Instalar as dependências

Com o ambiente ativado, instale as bibliotecas:

```
git clone https://github.com/SEU_USUARIO/web-beautyparlor-management.git cd
web-beautyparlor-management

python -m venv venv
```

Sistema Operacional	Comando no Terminal
Linux / macOS	source venv/bin/activate
Windows (CMD)	venv\Scripts\activate
Windows (PowerShell)	venv\Scripts\Activate.ps1

```
pip install -r requirements.txt
```

Passo 1 — Conectar ao repositório principal

Garanta que sua cópia local enxerga e baixa as atualizações do repositório principal.

```
# Fazer apenas uma vez git remote add upstream
https://github.com/perinotti/web-beautyparlor-management.git # Sempre que for começar um
trabalho novo, atualize sua develop local git checkout develop git pull upstream develop
```

Passo 2 — Criar uma branch para o seu trabalho

Nunca trabalhe diretamente nas branches **develop** ou **master**. Crie uma branch com nome descritivo seguindo o padrão: **tipo/descricao-curta**

```
git checkout -b fix/crash-pagamento-credito # Commit 1 git checkout -b
fix/imports-deprecados # Commit 2 git checkout -b fix/remove-create-all-alembic # Commit
3 git checkout -b fix/status-http-401-login # Commit 4
```

Passo 3 — Fazer as alterações e testar

Faça apenas as alterações do seu commit. Use o checklist do card do Trello como roteiro. Se fechou o terminal, reative o ambiente virtual antes de testar:

Sistema Operacional	Comando
Linux / macOS	source venv/bin/activate
Windows	venv\Scripts\activate

```
uvicorn main:app --reload
```

Passo 4 — Fazer o commit

Use a mensagem exatamente como definida no Plano de Ação. O padrão é o **Conventional Commits**. Evite **git add .** — adicione só os arquivos alterados.

```
git add database.py models.py routers/painel.py git commit -m "fix: corrige imports
deprecados, relacionamento quebrado e import duplicado"
```

Passo 5 — Enviar para o GitHub

```
git push origin fix/imports-deprecados
```

Passo 6 — Abrir um Pull Request (PR)

1. Acesse seu repositório no GitHub e clique em **"Compare & pull request"**.
2. **Atenção:** certifique-se de que o destino é a branch **develop** do repositório principal, não master.
3. Preencha o título com a mensagem do commit e a descrição usando o template:

```
## O que foi alterado - database.py: movido import de `declarative_base` - models.py:
adicionado `back_populates='vendas'` - routers/painel.py: removido import duplicado de
defaultdict ## Por que foi alterado Correções de qualidade identificadas no mapeamento
inicial. ## Como testar 1. Rodar `uvicorn main:app --reload` 2. Verificar que nenhum
aviso de deprecacao aparece no terminal 3. Navegar pelo painel e verificar que produtos
carregam ## Migration necessaria? [ ] Sim - rodar `alembic upgrade head` antes de testar
[x] Nao
```

4. Confirme que o destino é **perinotti/web-beautyparlor-management** → branch **develop** e clique em **"Create pull request"**.

Para o dono do repositório

Como testar um Pull Request antes de aceitar

1. Buscar a branch do colega localmente:

```
git fetch origin git checkout nome-da-branch-do-colega
```

2. Atualizar o banco se houver migration nova:

```
alembic upgrade head
```

3. Rodar o servidor e testar:

```
source venv/bin/activate # ou venv\Scripts\activate no Windows uvicorn main:app --reload
```

4. Se tiver problema: comente no PR apontando o que encontrou. O colega corrige na mesma branch e o PR atualiza automaticamente.

5. Se estiver tudo certo: clique em **"Merge pull request"** → **"Confirm merge"**.

Checklist de revisão antes de aceitar qualquer PR

- As alterações correspondem ao commit descrito no Plano de Ação?
- Nenhuma outra parte do código foi modificada sem necessidade?
- A mensagem do commit segue o padrão Conventional Commits?
- O servidor sobe sem erros no terminal?
- As páginas afetadas funcionam sem erro 500?
- Se teve migration, o alembic upgrade head rodou sem erro?
- O alembic current mostra (head) depois?

Ordem recomendada para aceitar os PRs

Respeite as dependências definidas no documento **"Dependências Importantes"**:

1. Mergear Commits 1, 2 e 3 antes de qualquer coisa da Fase 2.
2. Na Fase 2, mergear o commit de remoção do create_all antes do commit de magic strings.
3. Nunca mergear os commits de sessão e magic strings ao mesmo tempo — um por vez.

Como promover develop para master (ao final de uma fase)

Após todos os commits de uma fase estarem mergeados em develop e testados:

```
git checkout master git pull upstream master git merge develop git push origin master
```

Ou pelo GitHub: abra um PR de develop para master.

Como atualizar o repositório local após um merge

```
git checkout develop git pull upstream develop
```

Resumo do fluxo em uma linha: pull upstream develop → nova branch → alterar → testar → commit → push → PR para develop → revisão → merge